

Lab 1. Ticket enhancer

Saturday walkthrough. Claude Code plugin.

A vague GitHub issue in. A ready contract out.

25 minutes of typing. Fork setup is already done.

Work from `labs/lab1_enhancer`. Not the repo root.



What you will build

A Claude Code plugin that grooms every open draft ticket in **your** fork, one poll at a time.

PIECE	PATH YOU CREATE
Judge agent	<code>.claude/agents/enhancer-judge.md</code>
Doer agent	<code>.claude/agents/enhancer-doer.md</code>
Field check	<code>.claude/skills/enhancer-loop/scripts/check_fields.py</code>
Stop check	<code>.claude/skills/enhancer-loop/scripts/check_stop.py</code>
Orchestrator skill	<code>.claude/skills/enhancer-loop/SKILL.md</code>

Already in the folder: `Taskfile.yml`, `bin/*.sh`, `config.json.example`, `.claude/settings.json`, four prompt files.

Why this lab exists

A prompt that grooms a ticket once is a demo. A loop that polls GitHub, drafts, judges, and stops is a product.

- Nobody sits in chat driving it.
- The judge cannot edit the ticket it grades.
- Ready is arithmetic, not a model claim.
- `task poll-forever` is a seminar lie. Production is an event. See `GITHUB-ACTIONS.md`.

Final outcome. T900, T901, T902 exist as GitHub issues. One `task run` grooms the stubs. A human LGTM on a green rubric sets `ready` and `loop: implementer`.

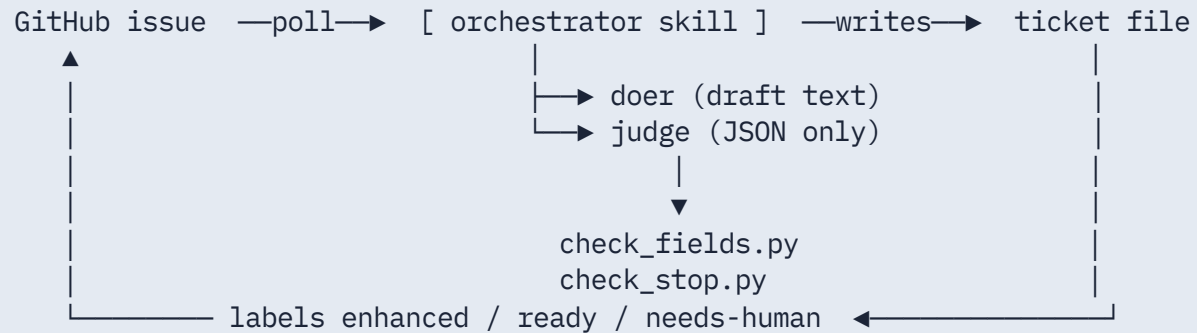
Learning objectives

After this lab you can:

- Configure a Claude Code plugin with two subagents and one skill
- Implement a judge that holds **no write tool**
- Implement a doer that returns a draft as text, not as a file write
- Validate ready with `check_fields.py`, not with the model's own JSON
- Stop on budget or a repeated signature with `check_stop.py`
- Troubleshoot a poll that labels `enhanced` without rewriting the ticket
- Point the same loop at GitHub Actions later, without changing the exits

Starting architecture

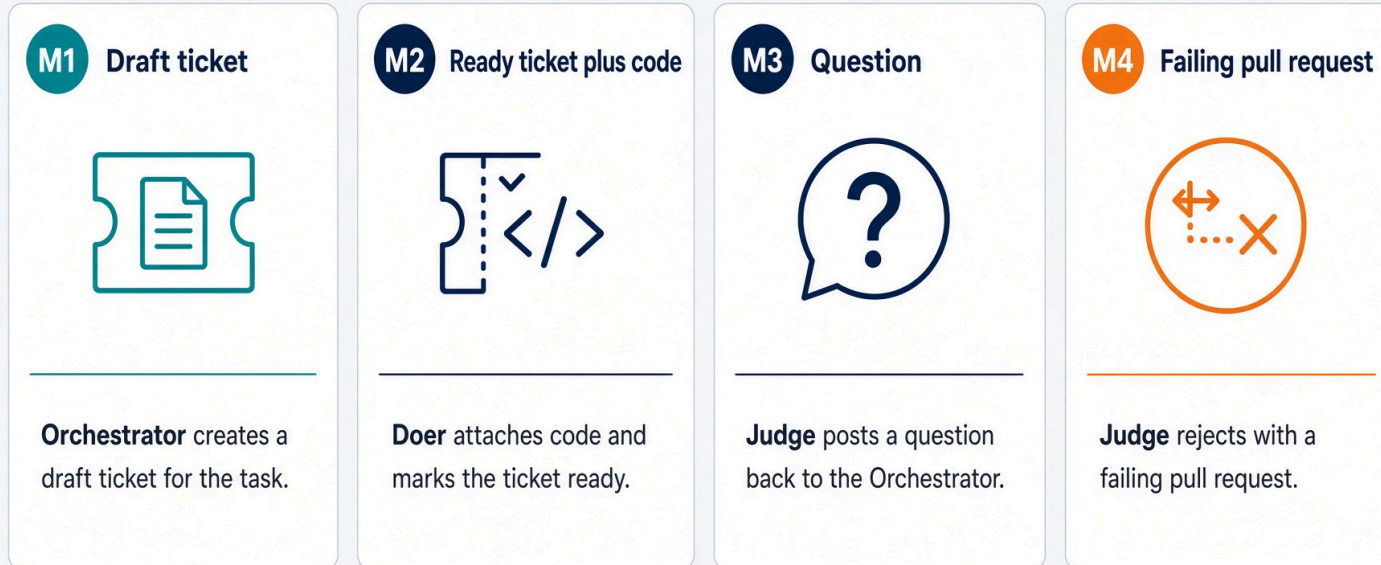
Components that already exist (gray). Components you create (navy).



Trigger lives **outside**: `task run`, `task poll-forever`, `/loop`, or **Actions**.
The skill runs **one step and exits**.

Four objects, this hour is tickets

Same graph: Orchestrator plus Doer plus Judge



Same graph every hour. Module 1 swaps in a draft ticket.

Orchestrator writes. Doer drafts. Judge scores. Human says LGTM.

Prerequisites

NEED

GitHub account and `gh`

Claude Code

Fork of northwind-field-crm, in **your** account`task`

Working directory

CHECK

`gh auth status``claude --version``never SpillwaveSolutions``task --version``cd labs/lab1_enhancer`

```
cp config.json.example config.json
# fill fork_owner with your GitHub username
task clone
```

`task clone` **writes** `../../work/northwind-field-crm`. One clone for every module.

Expected: a git checkout under `work/`, not an error about `fork_owner`.

Step 1. Start Claude in this folder

```
cd labs/lab1_enhancer
claude
```

Paste **one prompt at a time** from `prompts/claude-code.md`.

`.claude/settings.json` already denies:

```
Write/Edit ../../loops/**
Write/Edit ../../scripts/**
Write/Edit ../../work/**/tests/**
```

You are not editing the CRM tests. You are not rebuilding a shared engine.
There is no `loops/` package. Duplicate code on purpose.

Step 2. Judge agent

What. Create `.claude/agents/enhancer-judge.md`.

Why. A judge that can edit the ticket can grade itself.

```
---
name: enhancer-judge
description: Reads one ticket file and reports which required fields for its kind have real content. Never writes anything, never grades its own d
tools: Read, Grep, Glob
---
```

Entire final message is one JSON object:

```
{"kind": "bug", "present_fields": ["title", "steps"]}
```

List only fields with real content, not a bare heading.

Required fields, by kind

KIND	REQUIRED
bug	title (8+ characters), steps, expected, actual, environment
feature	problem, proposal, value, criteria (2+ a test can fail)
ui	feature, plus wireframe

Classify from title and body: crash / error / fails / regression → bug. form / page / button / screen / layout → ui. Otherwise feature.

The judge reports `present_fields`. It does **not** compute `missing_fields`. That is Python's job.

Step 3. Doer agent

What. Create `.claude/agents/enhancer-doer.md`.

Why. The draft must be judged before it can reach the real ticket.

Same tools: `Read`, `Grep`, `Glob`. **No write.**

Investigate `app/` in the target repo. Use the latest human GitHub comment as the strongest signal. Where neither settles a field, fill a stated guess. A missing field blocks the ticket. A stated guess does not.

The orchestrator writes `tickets/<id>.enhancer-candidate.md` from the doer's text. The doer does not write that file itself.

Step 4. Deterministic field check

What. `.claude/skills/enhancer-loop/scripts/check_fields.py`

The Judge agent decides which fields have real content (a model judgment). This script decides whether that adds up to ready (a fact).

```
python3 .claude/skills/enhancer-loop/scripts/check_fields.py --demo
```

Expected:

```
check_fields: all demo assertions passed
```

It reads `{"kind", "present_fields"}` and prints `missing_fields` and `ready`. It ignores any `missing_fields` the caller also sent. Invented field names are dropped.

Step 5. Deterministic stop

What. `.claude/skills/enhancer-loop/scripts/check_stop.py`

A stop condition trusted to a model's own judgment is a stop condition a model can talk itself past.

Input: `round, budget, signature, previous_signature`.

Output: `{"stop": bool, "reason": str or null}`.

Stop when the signature repeats (not on round 0), or when `round + 1 >= budget` (budget is 3).

```
python3 .claude/skills/enhancer-loop/scripts/check_stop.py --demo
```

Expected: `check_stop: all demo assertions passed.`

Step 6. Orchestrator skill. The eight rules

```
/enhancer-loop --repo <path> [--ticket <id>] [--simulate-comment "<text>"]
```

Hard rules from the answer's `SKILL.md`:

- A missing comment does not stop you. Do not fetch comments until `check_fields.py` says ready.
- The `enhanced` label is not the work. Adding it without rewriting the ticket file is a failed poll.
- Seed stubs are never ready.
- An issue opened in the GitHub UI is a ticket. Materialize a local file.
- `ready` comes from `check_fields.py`, never from the judge's own claim, never from a label, never from a comment other than exact LGTM.

Issue lookup. Order is load-bearing

1. State file `.harness/last-enhancer-<id>.json` field `github_issue`
2. Ticket frontmatter `github_issue`
3. Title search `--state all` for `[<id>]`

Never `--state open`. Never `gh issue create` from `task run`. Creating tickets is `task create-test-tickets`.

Closed issue → stop, say so. None found → tell them to run `task create-test-tickets`.

Strict improvement and the marker

Candidate `missing_fields` must be a **proper subset** of the current ticket. "Not worse" is not enough.

On keep: copy candidate over the real ticket, `gh issue edit --body (frontmatter stripped), --add-label enhanced`.

On no-op: leave the file and the issue body untouched.

Either way: delete the candidate. Post one comment containing:

```
<!-- enhancer-loop -->
```

When you later read comments, skip any body that contains that marker. **Do not filter by author.** The loop runs as your `gh` account. An author filter would drop your own LGTM.

Three exits. No fourth.

EXIT	WHEN	LABELS
pass	newest human comment is exactly LGTM and the rubric is already green	ready. Frontmatter state: ready, loop: implementer. Delete the state file.
escalate	same missing_fields twice	needs-human
escalate	round budget spent (3)	needs-human

LGTM on a red rubric finalizes nothing.

enhanced is history. It stays after ready.

State file. Memory for a memoryless job

<repo>/.harness/last-enhancer-<id>.json

```
{
  "github_issue": 42,
  "last_comment_id": 1000000001,
  "round": 1,
  "previous_signature": ["criteria", "value"]
}
```

A scheduled Actions job has no chat transcript. This file is how the next poll resumes. Deleted on pass. Frontmatter `github_issue` outlives it.

Step 7. Seed tickets and run one poll

```
task create-test-tickets && task run --
```

ID	KIND	TITLE
T900	bug	Search crashes on an empty query
T901	ui	Add a notes field to the customer page
T902	feature	Export tasks to CSV

```
task run is claude -p "/enhancer-loop --repo {{.TARGET}}".
```

Expected narration: one line per ticket, `waiting` / `escalated` / `passed`. First poll grooms stubs with no comment. Seed stubs are never ready.

Expected result after poll 1

- Ticket files under `work/northwind-field-crm/tickets/` have real fields, not a title plus one sentence
- Label `enhanced` on issues that were actually rewritten
- A marked comment on those issues
- `.harness/last-enhancer-T900.json` (and T901, T902) exist
- `check_fields.py` still reports missing fields on a stub that was not improved

Second `task run` with no new human comment: no-op on a ticket whose candidate was not better. Must **not** post a second identical reply.

Human LGTM, then pass

Comment `LGTM` on GitHub as yourself, on a ticket `check_fields.py` already calls ready.

Next poll:

- label `ready`
- frontmatter `state: ready` and `loop: implementer`
- state file deleted

That is the handoff into Lab 2.

For the seminar:

```
task poll-forever --
```

`while true: task run; sleep poll_interval.` It never self-stops. `Ctrl-C` when you are done. Use `poll_interval: 30s` in `config.json` while testing. `10m` is production.

Validation checklist

- ☐ `check_fields.py --demo` passes
- ☐ `check_stop.py --demo` passes
- ☐ Judge YAML lists `Read`, `Grep`, `Glob` only
- ☐ Doer YAML lists the same, no `Write`
- ☐ One `task run` rewrites at least one stub and labels `enhanced`
- ☐ Loop comments contain `<!-- enhancer-loop -->`
- ☐ A second poll does not duplicate that comment
- ☐ LGTM on a green rubric sets `ready`
- ☐ LGTM on a red rubric does not

Prompt 5 in `prompts/claude-code.md` diffs your `.claude/` against `../../solutions/sol1_enhancer/.claude/`.

Troubleshooting

SYMPTOM	LIKELY CAUSE	RESOLUTION
Skill not found	ran <code>claude</code> from the repo root	<code>cd labs/lab1_enhancer</code>
<code>enhanced</code> label, body unchanged	orchestrator labeled without rewriting	failed poll. Rewrite, then label.
Loop ignores your LGTM	filtered comments by author	filter on the marker, not the author
Infinite comments	marker missing on posted bodies	add <code><!-- enhancer-loop --></code>
<code>gh issue create</code> during <code>task run</code>	prompt drift	never <code>create.task</code> <code>create-test-tickets</code> only.
Closed issue reused	searched <code>--state open</code> only	lookup uses <code>--state all</code>
Hang on poll-forever	expected	<code>ctrl-C</code> . It never self-stops.

Fall behind

Nobody is graded. Save the attempt first.

```
# Lab 1 is the only lab with a drop-in
cp -R ../../solutions/sol1_enhancer/.claude .claude
```

See `FALL-BEHIND.md`.

Other tools, after class, not now:

TOOL	ANSWER FOLDER
Codex	solutions/sol1_enhancer_codex
OpenCode	solutions/sol1_enhancer_opencode
Grok Build	solutions/sol1_enhancer_grok_build
Agent SDK	solutions/sol1_enhancer_agent_sdk (issue 118 family)
Deep Agents	solutions/sol1_enhancer_deep_agents

Production. Actions, not a laptop loop

`SPEC.md` said the workflow file did not exist yet. It does now, as a copy-me.

- **Notes:** `labs/lab1_enhancer/GITHUB-ACTIONS.md`
 - **Workflow:** `labs/lab1_enhancer/workflows/enhance-on-issue.yml`
-

Copy the YAML onto **your CRM fork**. Listen to `issues` opened / edited / labeled and `issue_comment` created.

Skip `<!-- enhancer-loop -->`. Concurrency per issue. Label `agent-in-progress`. Cap `AGENT_MAX_ATTEMPTS`.

The trigger moves. The exits stay. A workflow that merges has left the lab.

Recap

What we built. A plugin: judge, doer, two Python checks, one orchestrator skill that polls once and exits.

What you learned. Scope is a missing method. Ready is arithmetic. The interesting run is the one that stops.

Takeaways

1. Trigger outside, exits inside.
2. Judge has no write tool.
3. `check_fields.py` computes ready. The model does not.
4. Marker, not author, when reading comments.
5. `task poll-forever` is a seminar stand-in. Actions is the deploy.

Closing line. The loop is the product. The prompt is not.